

۸۱۰۱۰۱۴۰۸

۸۱۰۱۰۱۵۴۴

۸۱۰۱۰۱۴۹۲

۸۱۰۱۰۱۵۸۲

سیستم‌های نهفته بی‌درنگ

نام و نام خانوادگی اعضای گروه:

بابک حسینی محتشم

هانیتا نیک نسب

علیرضا کریمی

نرگس اسدی خوانساری



تمرین شماره ۱

۱ مقدمه

در این تمرین با شبیه‌ساز داده شده آشنا می‌شویم و با اضافه کردن سه پروتکل ارتباطی بین دستگاه‌های مختلف، شبیه‌ساز داده شده را کامل می‌کنیم.

ابتدا کتابخانه boost را استخراج کرده و در فایل CMakeLists مسیر درست را قرار می‌دهیم. سپس فولدر build را ایجاد کردیم و به صورت آزمایشی، شبیه‌ساز را کامپایل کردیم. با ایجاد اطمینان از نصب بودن وابستگی‌های لازم، به پیاده‌سازی پروتکل‌ها می‌پردازیم.

تقسیم‌بندی وظایف: در طول پروژه در صورت برخورد به مشکل آن را در گروه مطرح می‌کردیم و تمام اعضای گروه در حل آن مشارکت داشتند به همین دلیل هر نفر سهمی از هر قسمت پروژه دارد ولی تقسیم‌بندی اولیه‌ای که انجام دادیم و مطابق آن بیشترین سهمی که به هر شخص دادیم بدین صورت بود:

۱. بابک: کد و گزارش بخش اول

۲. نرگس: کد و گزارش بخش دوم

۳. هانیتا: کد و گزارش بخش سوم

۴. علیرضا: پاسخ به سوالات و کمک در کد و گزارش بقیه قسمت‌ها

۲ پیاده‌سازی پروتکل آدرس‌محور (I2C)

۱.۲ توضیحات مسئله

پروتکل I2C (Inter-Integrated Circuit) یک پروتکل باس برای ارتباط سریال در محدوده کوتاه بین دستگاه‌های الکترونیکی مانند میکروکنترلر با سنسورها است. این پروتکل به دلیل سادگی و نیاز به سیم‌کشی کم به صورت گسترده‌ای استفاده می‌شود.

این پروتکل به چهار پایه روی برد سنور و میکروکنترلر نیاز دارد که بدین صورت هستند:

□ VDD: همان منبع تغذیه در سیستم‌های الکترونیکی است.

□ GND: همان زمین مشترک در سیستم‌های الکترونیکی است.

□ SCL: باس کلاک مشترک بین دو دستگاهی که در ارتباط هستند.

□ SDA: باس داده که دو دستگاه از طریق این باس داده و پیام‌های کنترلی را انتقال می‌دهند.

کارکرد این پروتکل به صورت کلی بدین شکل است که ابتدا میکروکنترلر با فرستادن سیگنال در SDA شروع ارتباط را مشخص می‌کند. سپس آدرس مورد نظر را در کلاک بعد روی SDA به همراه یک بیت که مشخص کننده‌ی خواندن (۱) یا نوشتن (۰) است می‌فرستد. دستگاه فرستنده داده‌ها را در هر کلاک یک بایت ارسال می‌کند و گیرنده با دریافت هر بایت، یک ACK ارسال می‌کند. هنگامی که میکروکنترلر SDA سیگنال دیگری را ارسال کند، ارتباط پایان می‌یابد. در این پروژه نسخه ساده شده‌ای از این پروتکل را پیاده می‌کنیم. در این نسخه، ابتدا میکروکنترلر آدرس سنسور مقصد را روی لینک SDA می‌نویسد و هر سنسور آدرس مختص خود را دارد. بدین ترتیب سنسور با مشاهده آدرس مربوط به خود، یک بیت ۱ ACK ارسال می‌کند و هر کلاک یک بایت داده را روی SDA قرار می‌دهد.

۲.۲ سوالات:

(۱) با توجه به اینکه I2C (نسخه‌ی اصلی که در درس با آن آشنا شده‌اید) یک پروتکل باس است در صورت اتصال بیشتر از یک سنسور با آدرس‌های مشابه چه اشکالی پیش می‌آید؟

از آنجایی که آدرس قرار گرفته بر SDA برای خواندن داده، مخصوص هر دو سنسور است هر دو بیت ACK را قرار می‌دهند و در هر کلاک به طور همزمان داده‌های خود را روی باس SDA قرار می‌دهند که باعث ایجاد تداخل و نرسیدن داده درست به میکروکنترلر می‌شود.

(۲) چهار مورد از اشکالات کلی این پروتکل ساده‌سازی شده نسبت به آنچه از I2C در درس آموخته‌اید بیان کنید. نسخه ساده شده ایراداتی نسبت به نسخه اصلی پروتکل دارد که چهار مورد از آن را می‌توان نام برد:

۱. چون پس از هر بایت داده بیت ACK ارسال نمی‌شود، اگر گیرنده نتواند بایت ارسال شده را به هر دلیلی مانند کند بودن سرعت پردازش داده نتواند دریافت کند، آن بایت داده از دست می‌رود و گیرنده با دریافت بایت‌های بعدی آن‌ها را به جای آن بایت از دست داده شده در نظر می‌گیرد.

۲. در این نسخه ساده شده میکروکنترلر تنها می‌تواند داده دریافت کند و نمی‌تواند داده‌ای روی سنسور بنویسد.

۳. در نسخه اصلی، ابتدا میکروکنترلر یک سیگنال می‌فرستد که شروع ارتباط را نشان می‌دهد و سپس آدرس مورد نظر را قرار می‌دهد و این باعث مشخص شدن آغاز ارتباط است ولی در این نسخه ساده شده تنها با قرار گرفتن آدرس روی SDA ارسال داده شروع می‌شود و بدین ترتیب ممکن است در حین فرآیند ارسال داده در صورتی که چندین سنسور وجود داشته باشند، به اشتباه فرض کنند داده ارسال شده آدرس آن‌ها است که میکروکنترلر قرار داده است.

۴. این پروتکل ساده شده روشی برای آنکه میکروکنترلر بداند داده‌ها تمام شده‌اند در نظر نگرفته است.

(۳) کارکرد کلاس‌های `ByteStream`، `ByteVector` و تابع `getBytes` را توضیح دهید.

□ `ByteStream`: یک کلاس `template` است که با کمک عملیات شیفت، بایت‌های دریافتی را می‌چسباند تا به اندازه شیء `template` برسد و در آن صورت این مقدار را می‌تواند برگرداند.

□ `ByteVector`: این کلاس مانند آرایه‌ای از بایت‌ها عمل می‌کند و دسترسی به هر یک از بایت‌ها را فراهم می‌کند.

□ `getBytes`: سه تابع `overload` شده‌اند که با دریافت داده، بایت مشخصی از آن را استخراج می‌کنند.

۳.۲ پیاده‌سازی

۴.۲ `main.cpp`

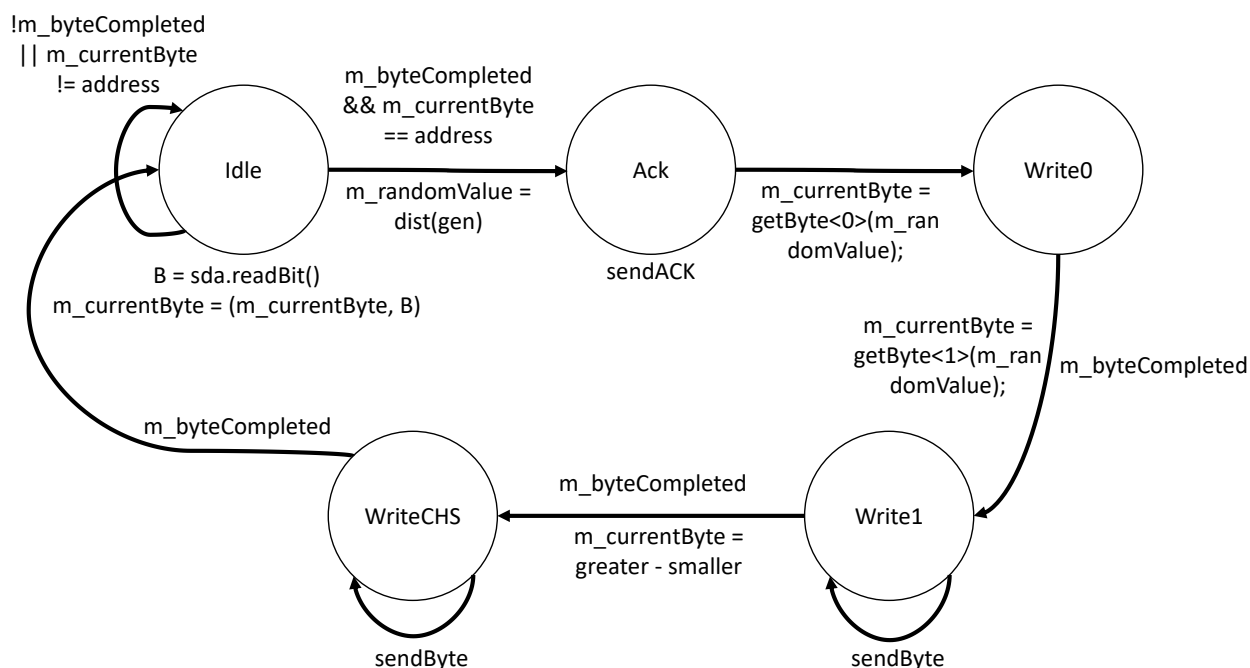
نحوه پیاده‌سازی این تمرین مشابه پروژه‌های سخت‌افزاری انجام شد. ابتدا فایل‌هایی که در پیاده‌سازی این سوال به کار گرفته شدند را توضیح می‌دهیم. فایل اصلی برنامه که در آن ماژول‌ها ایجاد می‌شوند و به یکدیگر متصل می‌شوند، `main.cpp` است. در این فایل دستگاه‌های میکروکنترلر و سنسور ایجاد می‌شوند و چهار پایه بیان شده از دو دستگاه به هم وصل می‌شوند. سپس هر دو شروع به کار می‌کنند. این عملیات در فایل داده شده از قبل نوشته شده بود و نیازی به ایجاد تغییر نبود.

۵.۲ کلاس I2C

این کلاس در کلاس‌های متناظر میکروکنترلر و سنسور تعریف شده است. برای اضافه کردن پروتکل I2C به این دستگاه‌ها، باید کلاس I2C که در این کلاس‌ها تعریف شده است را پیاده کنیم. این کلاس شامل چهار تابع `init`، `write`، `read` و `run` است. تابع `init` با منظور اجرا در آغاز برای مقداردهی‌های اولیه را دارد. تابع `write` برای نوشتن در دستگاه متقابل توسط این پروتکل است که در این نسخه ساده شده پروتکل، تنها داده‌هایی به منظور خواندن ارسال می‌شوند و نیازی به پیاده‌سازی این تابع نیست. تابع بعدی `read` است که به منظور خواندن داده طبق این پروتکل وجود دارد. این تابع تنها در میکروکنترلر پیاده شده است چراکه قرار نیست سنسور داده‌ای از میکروکنترلر بخواند. تابع نهایی `run` است. این تابع هر کلاک فراخوانی می‌شود و پروتکل I2C در بدنه این تابع پیاده می‌شود. حال به نحوه پیاده‌سازی این تابع در هر یک از دو دستگاه می‌پردازیم.

۶.۲ کلاس VL530X

این کلاس عملکرد سنسور را شبیه‌سازی می‌کند. برای پیاده‌سازی عملکرد، مطابق الگوی پیاده‌سازی در سخت‌افزار با مفهوم استیت ماشین پیاده می‌کنیم.



شکل ۱: استیت دیاگرام سنسور

دیگرام حالات سنسور در **نگاره ۱** آمده است. سنسور در ابتدا در حالت Idle است و مدام بیت‌های ورودی را دریافت می‌کند و بررسی می‌کند آیا تجمیع بیت‌های اخیر آدرس مربوط به سنسور که برای این سنسور برابر 0x29 است را تشکیل می‌دهد یا خیر. هر وقت که آدرس معتبری دریافت کرد عدد تصادفی بین ۰ تا ۴۰۰۰ به عنوان مقدار اندازه‌گیری شده ذخیره می‌کند و به حالت Ack می‌رود. در این هنگام میکروکنترلر منتظر دریافت سیگنال تایید است که در این حالت سنسور این سیگنال را ارسال می‌کند. سپس به حالت Write0 می‌رود. در این حالت بایت اول عدد تصادفی در هشت مرحله ارسال می‌شود و با رفتن به حالت Write1 به طور مشابه در هشت مرحله بایت دوم ارسال می‌شود. در نهایت به حالت WriteCHS می‌رود که Checksum را به صورت حاصل تفریق بایت کوچکتر از بایت بزرگتر به دست می‌آورد و آن را هم در هشت مرحله ارسال می‌کند و با پایان یافتن این چرخه، سنسور به حالت Idle باز می‌گردد تا بار دیگر منتظر دریافت آدرس متناظر خود باشد.

برای کارکرد درست تعدادی متغیر تعریف شده‌اند که آن‌ها را تعریف می‌کنیم. متغیر بولین m_byteCompleted نشان‌دهنده آن است که یک بایت داده دریافت یا ارسال شده است و نشان‌دهنده پایان هشت مرحله بیان شده است. متغیر m_currentByte بایت کنونی که در حال دریافت یا ارسال شدن است را نشان می‌دهد. متغیر m_state همان وضعیت سنسور را نشان می‌دهد. متغیر bitIndex تعداد بیت‌هایی از m_currentByte که ارسال یا دریافت شده‌اند را نشان می‌دهد.

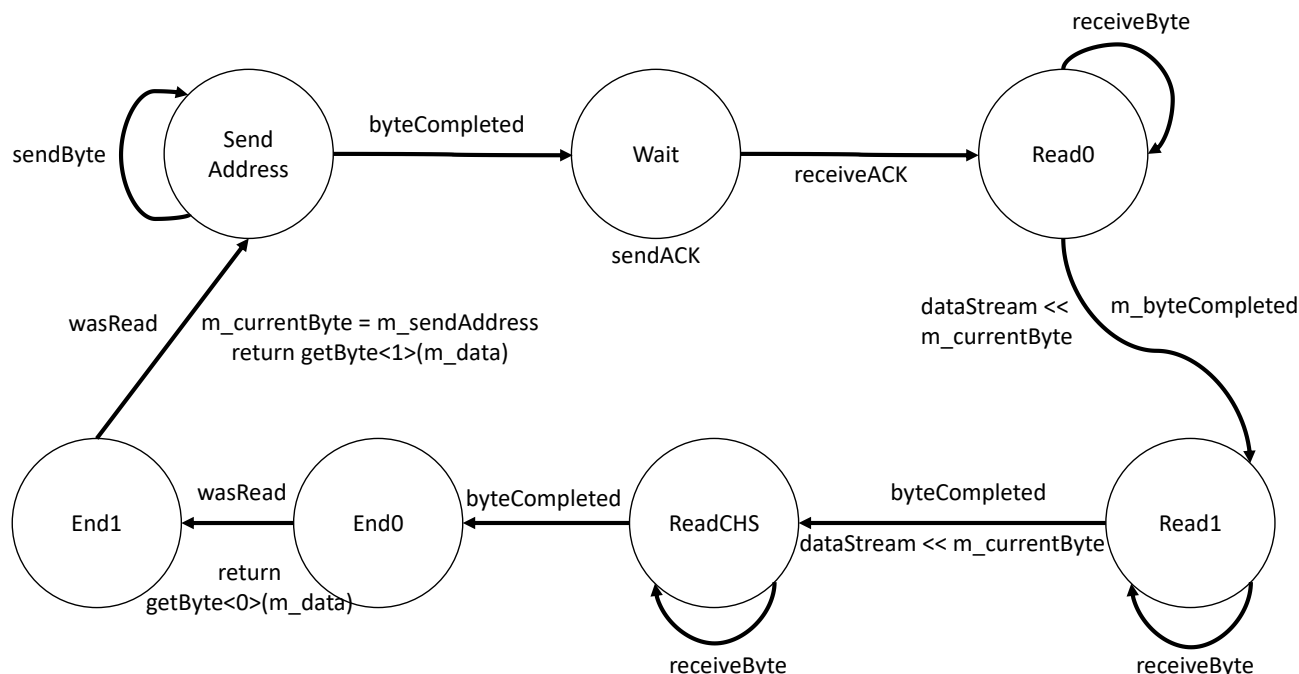
۷.۲ کلاس Esp8266

این کلاس میکروکنترلر را شبیه‌سازی می‌کند و اجزای آن مشابه سنسور است.

دیگرام حالات میکروکنترلر در **نگاره ۲** آمده است. در ابتدا در وضعیت SendAddress، این دستگاه آدرس سنسور مورد نظر را ارسال می‌کند. سپس به وضعیت Wait می‌رود و منتظر می‌ماند تا سنسور سیگنال Ack را ارسال کند. با دریافت این سیگنال، به وضعیت Read0 می‌رود و بایت اول داده را می‌خواند و پس از رفتن به وضعیت Read1 بایت بعدی خوانده می‌شود. سپس به وضعیت ReadCHS می‌رود تا بایت مربوط به Checksum خوانده شده و درستی داده‌ها تایید شود. در صورت درست بودن داده‌ها، به وضعیت End0 می‌رود و آن‌قدر آنجا می‌ماند تا طبق Sketch بایت اول داده مورد استفاده قرار گیرد و در نهایت در End1 هم بایت دوم مورد استفاده قرار می‌گیرد و به وضعیت SendAddress برمی‌گردد.

۸.۲ Sketchs

برای هر کدام از سنسور و میکروکنترلر یک Sketch تعریف می‌کنیم که نحوه استفاده از این دستگاه‌ها و داده‌ها را مشخص می‌کنند. sketch مربوط به سنسور به جز مقداردهی اولیه در ابتدا و سپس اجرا در هر کلاک، کار دیگری صورت نمی‌گیرد. میکروکنترلر در ابتدا مقداردهی اولیه می‌شود و سپس در هر کلاک تابع isDataAvailable از



شکل ۲: استیت دیاگرام میکروکنترلر

میکروکنترلر صدا زده می‌شود تا در صورتی که میکروکنترلر به وضعیت End0 رسیده است و داده‌ای آماده به خواندن دارد، این داده خوانده شود و سپس چاپ شود.

۹.۲ تکمیل پیاده‌سازی پروتکل I2C و انتقال از بیت به بایت

در پیاده‌سازی اولیه پروتکل I2C که در گزارش توضیح داده شد، ارتباط بین میکروکنترلر و سنسور به صورت بیت به بیت انجام می‌شد. در این روش، متغیرهایی مانند `m_bitIndex` برای شمارش بیت‌های ارسال یا دریافت شده در هر بایت داشتیم و در هر کلاک یک بیت پردازش می‌شد. پس از بحث و بررسی، تصمیم گرفتیم پروتکل را برای سادگی و کارایی بیشتر به صورت بایت به بایت تغییر دهیم. در روش جدید:

□ **حذف متغیر `m_bitIndex`** در روش قبلی باید تعداد بیت‌های ارسال یا دریافت شده از هر بایت را می‌شمردیم تا بدانیم چه زمانی یک بایت کامل شده است. با تغییر به روش بایتی، توابع `read()` و `write(Byte)` مستقیماً یک بایت کامل را در یک سیکل پردازشی انتقال می‌دهند و دیگر نیازی به شمارش بیت‌ها نیست.

□ **ساده‌سازی ماشین حالت:** در نسخه بیتی، هر حالت مانند `Write0` باید هشت بار (برای هشت بیت) اجرا

می‌شد تا یک بایت کامل منتقل گردد. اما در نسخه بایتی، هر حالت دقیقاً یک بار اجرا می‌شود و کل بایت را یکجا انتقال می‌دهد. این تغییر، ماشین حالت را بسیار ساده‌تر و خواناتر کرده است.

□ **کاهش تعداد متغیرهای کمکی:** علاوه بر `m_bitIndex` دیگر نیازی به متغیر `m_byteCompleted` برای تشخیص پایان انتقال یک بایت نداشتیم. کل فرآیند انتقال داده با تعداد حالات کمتر و بدون نیاز به متغیرهای واسط انجام می‌شود.

دلیل نگه‌داشتن نسخه بیتی در گزارش: با وجود آنکه نسخه بایتی ساده‌تر و بهینه‌تر است، اما نسخه بیت به بیت مزیت مهمی در فرآیند دیباگ داشت. از آنجایی که در روش بیتی، تک‌تک بیت‌های رد و بدل شده بین دو دستگاه را چاپ می‌کردیم، می‌توانستیم به راحتی هر بیت را رهگیری کرده و از درستی توالی بیت‌ها اطمینان حاصل کنیم. این قابلیت هنگام یافتن باگ‌هایی مانند اشتباه گرفتن بیت آخر آدرس با بیت ACK (که در بخش ارزیابی توضیح داده شد) بسیار حیاتی بود. بنابراین، با وجود تغییر نهایی به نسخه بایتی در کد، در گزارش توضیحات بر اساس نسخه بیتی نگه داشته شد تا فرآیند عیب‌یابی و چالش‌های آن نیز مستند گردد.

۱۰.۲ ارزیابی

پس از پیاده‌سازی، برای اطمینان از درستی مقادیر چاپ شده در Sketch میکروکنترلر را با مقادیر چاپ شده در وضعیت ابتدای سنسور که داده تولید می‌شود مقایسه کردیم و با یکسان بودن آن‌ها از درستی پیاده‌سازی اطمینان حاصل کردیم.

همچنین در طول پیاده‌سازی با چاپ کردن بیت‌های رد و بدل شده بین دو دستگاه، مشکلات و باگ‌ها را رفع کردیم. یکی از مشکلاتی که رفع آن زمان زیادی برد، این بود که به دلیل آنکه داده از هر دو طرف روی لینک SDA منتقل می‌شود، برای وقتی میکروکنترلر، آدرس را ارسال می‌کرد، بیت آخر آدرس را به اشتباه به عنوان ACK در نظر می‌گرفت و باعث می‌شد وضعیت آن جلوتر از سنسور باشد که با قرار دادن سیگنال Z پس از اتمام نوشتن هر دستگاه که باعث رها کردن لینک می‌شود این مشکل را رفع کردیم.

۳ پیاده‌سازی پروتکل Full Duplex (USART)

۱.۳ توضیحات مسئله

پروتکل USART برای ارتباط سریال دو دستگاه بدون وجود کلاک مشترک بین‌شان طراحی شده است. پروتکل USART به صورت دوطرفه یا Full Duplex است یعنی به طور همزمان هر دو دستگاه می‌توانند به یکدیگر داده ارسال کنند. در این پروتکل چهار پایه وجود دارد:

□ VDD: همان منبع تغذیه در سیستم‌های الکترونیکی است.

□ GND: همان زمین مشترک در سیستم‌های الکترونیکی است.

□ TX (Transmit): این پایه برای آن است که سنسور داده را روی آن بنویسد و میکروکنترلر از آن بخواند.

□ RX (Receive): این پایه هدفی عکس پایه قبلی دارد یعنی میکروکنترلر روی آن می‌نویسد و سنسور از آن می‌خواند.

در این پروتکل فرض می‌شود لینک در حالت عادی روی مقدار ۱ قرار دارد و ارسال داده در واحدهایی به نام فریم صورت می‌گیرد. هر فریم از اجزای زیر تشکیل شده است:

۱. Start Bit: با قرار دادن مقدار ۰ برخلاف مقدار حالت عادی که ۱ است، فرستنده اعلام شروع ارسال داده می‌کند.

۲. Data Bits: سپس ۸ بیت یا یک بایت از داده با شروع از کم‌ارزش‌ترین بیت شروع به ارسال می‌شوند.

۳. Parity Bit: از این بیت برای مشخص کردن زوجیت داده و تشخیص خطا در داده‌های دریافت شده استفاده می‌شود.

۴. Stop Bit: در پایان یک یا چند بیت با مقدار ۱ ارسال می‌شوند تا پایان فریم مشخص شود و گیرنده آماده دریافت فریم بعدی بشود.

در این تمرین بیت زوجیت را پیاده‌سازی نمی‌کنیم و فرض می‌کنیم داده‌ها بدون خطا ارسال و دریافت می‌شوند. نکته‌ای که در ارتباط با پیاده‌سازی این پروتکل وجود دارد آن است که به دلیل عدم وجود کلاک مشترک بین دو دستگاه، برای آنکه بتوان داده‌ها را بدون مشکل و به درستی دریافت کرد، باید هر دو دستگاه دارای کلاک درونی با Baud rate یکسان باشند یعنی نرخ ارسال و دریافت داده از روی پایه‌های TX و RX باید یکسان باشد تا هر دو دستگاه در نرخ زمانی یکسانی بیت‌ها را ارسال و دریافت کنند.

۲.۳ سوالات:

۱) توضیح دهید چرا بیت‌های شروع و پایان با بیت‌های داده اشتباه گرفته نمی‌شود؟
از آنجا که Baud rate کلاک دو دستگاه فرستنده و گیرنده یکسان است پس از آنکه فرستنده بیت صفر روی خط گذاشت و گیرنده آن را دریافت کرد، هر دو در نرخ زمانی یکسانی برای خواندن و نوشتن به خط می‌روند. بدین ترتیب پس از آنکه گیرنده به خواندن اولین بیت داده می‌رود می‌تواند اطمینان حاصل کند که به دلیل ارسال در نرخ یکسان در ارسال فرستنده، بیتی که می‌خواند برای داده است و در پایان هم با خواندن هر ۸ بیت و دریافت ۱ نهایی می‌تواند مطمئن باشد این بیت توقف است.

۲) با توجه به نبود کلاک توضیح دهید این پروتکل چه طور داده‌ها را تشخیص می‌دهد.
در پروتکل USART واقعی، دو دستگاه با Baud Rate یکسان کار می‌کنند. گیرنده با همان نرخ که فرستنده بیت‌ها را ارسال می‌کند، آن‌ها را نمونه‌برداری می‌کند. بیت شروع (Start Bit) به عنوان نقطه‌ی همگام‌سازی عمل می‌کند و به گیرنده می‌گوید که از کجا شروع به خواندن کند. حتی اگر Baud Rate دو دستگاه اختلاف کمی داشته باشد، به دلیل کوتاه بودن طول فریم (۸ بیت داده)، این اختلاف فرصت انباشته شدن پیدا نمی‌کند و با رسیدن بیت توقف (Stop Bit) و سپس بیت شروع فریم بعدی، همگام‌سازی مجدداً تنظیم می‌شود.

نکته قابل توجه در این شبیه‌ساز آن است که به دلیل وجود بافر (BitQueue) در لینک‌های ارتباطی (کلاس Transmitter)، حساسیت به اختلاف فرکانس حتی کمتر از دنیای واقعی است. در این سیستم، بیت‌های ارسالی در بافر ذخیره می‌شوند و گیرنده آن‌ها را به ترتیب و با سرعت پردازش خود از بافر می‌خواند. این مکانیزم باعث می‌شود که حتی اگر دو دستگاه با Baud Rate‌های متفاوت کار کنند، تا زمانی که نرخ تولید داده از نرخ مصرف آن بیشتر نشود، داده‌ها بدون خطا منتقل شوند. این یک ویژگی شبیه‌ساز است که انعطاف‌پذیری بیشتری نسبت به سخت‌افزار واقعی فراهم می‌کند.

۳) توضیح دهید آیا امکان پیاده‌سازی پروتکل USART به صورت نرم‌افزاری (با استفاده از پین‌های دیجیتال) در میکروکنترلرهایی که منجر به سخت‌افزار مربوطه نیستند امکان‌پذیر است؟ (به عنوان مثال این امکان برای I2C وجود دارد) دلایل خود را کامل شرح دهید.

در پیاده‌سازی نرم‌افزاری باید تایمری را ست کرد تا با نرخ ثابت و برابر در هر دو دستگاه فراخوان سیستمی دهد و یک بیت از داده ارسال یا دریافت شود. این کار امکان‌پذیر است ولی نسبت به پیاده‌سازی سخت‌افزاری معایب و چالش‌هایی دارد مانند آنکه نرخ کارکردن تایمر در نرم‌افزار محدودتر از پیاده‌سازی سخت‌افزاری است. همچنین به دلیل سرآیند فراخوانی سیستمی این نرخ را حتی باید مقدار کمتری هم در نظر گرفت. در نهایت اینکه باید وقفه‌های دیگر و عوامل دیگری که ممکن است باعث شوند در زمان مناسب بیت مورد نظر ارسال یا دریافت نشود را هم در نظر گرفت چرا که برای کارکرد درست این الگوریتم نیاز به نرخ ارسال و دریافت با اختلاف کمی در دو دستگاه است.

۳.۳ پیاده‌سازی

۴.۳ main.cpp

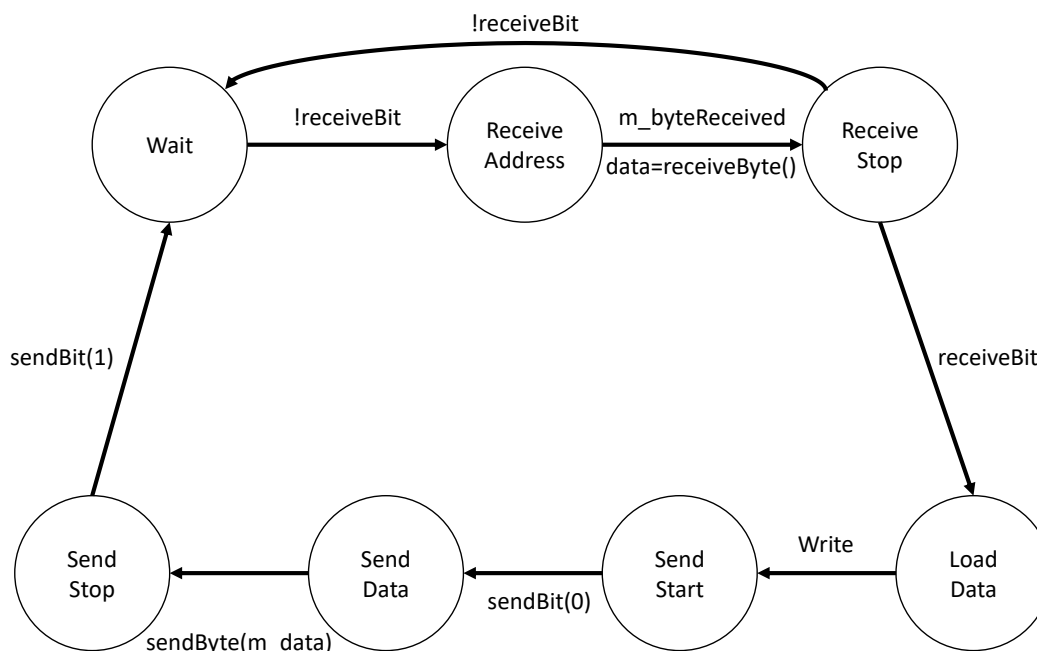
در فایل اصلی، یک نمونه از کلاس Usb ایجاد کردیم و پایه‌های آن را به میکروکنترلر متصل کردیم. از آنجایی که پایه tx در میکرو و rx در یواس‌بی تنها برای ارسال داده استفاده می‌شوند و نه دریافت آن بنابراین قابلیت خواندن از آن‌ها را در دستگاه مربوط تعیین کردیم. در نهایت با ساختن اسکچ هارد دیسک مرتبط با یواس‌بی فعالیت آن را شروع کردیم.

۵.۳ کلاس USART

این کلاس در کلاس‌های متناظر میکروکنترلر و یواس‌بی تعریف شده است. این کلاس شامل سه تابع write، read و init است. تابع write برای نوشتن در دستگاه متقابل توسط این پروتکل است که میکروکنترلر با استفاده از آن آدرس و یواس‌بی با استفاده از آن داده را می‌نویسد. تابع بعدی read است که به منظور خواندن داده طبق این پروتکل وجود دارد. میکروکنترلر با کمک آن داده ارسال شده را می‌خواند و یواس‌بی هم با این تابع آدرس را دریافت می‌کند. تابع نهایی run است. این تابع هر کلاک فراخوانی می‌شود و پروتکل USART در بدنه این تابع پیاده می‌شود. حال به نحوه پیاده‌سازی این تابع در هر یک از دو دستگاه می‌پردازیم.

۶.۳ کلاس Usb

این کلاس را در پوشه Comm ایجاد کردیم که متناظر یواس‌بی و همان دستگاهی است که میکرو کنترلر قصد دارد طبق پروتکل USART داده‌ای از آن بخواند. در ادامه نحوه کارکرد ماشین حالت این دستگاه را بررسی می‌کنیم. دیگرام حالات یواس‌بی در **نگاره ۳** آمده است. ابتدا یواس‌بی در حالت Wait شروع است و در این حالت می‌ماند تا وقتی که سیگنال صفر را از tx دریافت کند و با دریافت آن به حالت ReceiveAddress می‌رود. در این حالت یواس‌بی داده را دریافت می‌کند و با رفتن به حالت ReceiveStop بیت پایان فریم را هم دریافت می‌کند و به حالت LoadData می‌رود. البته ممکن است به جای مقدار یک که نشان دهنده بیت توقف است، مقدار صفر دریافت شود که در آن صورت یواس‌بی متوجه می‌شود داده ورودی را به اشتباه دریافت کرده است و ضمن چاپ پیغام خطا به حالت ابتدا بازمی‌گردد. یواس‌بی در حالت LoadData می‌ماند تا آنکه در اسکچ دستور خواندن آدرس دریافت شده با فراخوانی تابع read انجام شود و پس از آن با دستور write دستور به نوشتن داده متناظر به آدرس داده شده در rx داده می‌شود و حالت یواس‌بی به SendStart تغییر می‌کند تا بیت آغاز فریم را ارسال کند. سپس در دو حالت SendData و SendStop به ترتیب داده و بیت توقف فریم را ارسال می‌کند و به حالت ابتدایی برمی‌گردد.



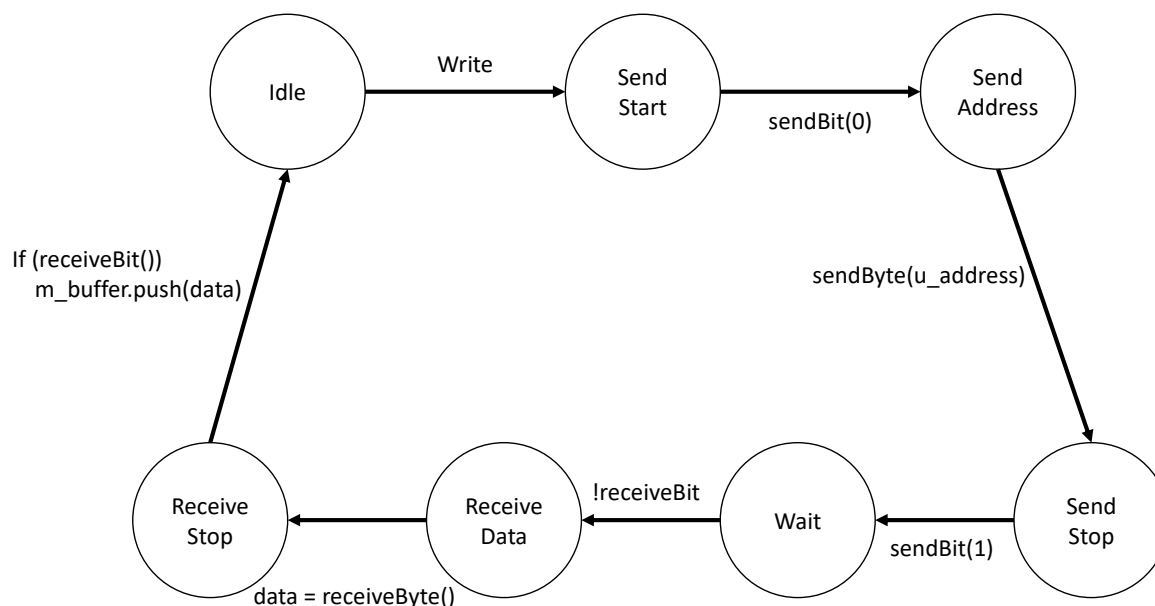
شکل ۳: استیت دیاگرام سنسور

۷.۳ کلاس Esp8266

دیاگرام حالات میکروکنترلر در **نگاره ۴** آمده است. میکروکنترلر در حالت Idle شروع به کار می‌کند و آن‌قدر آنجا می‌ماند تا در اسکچ تابع write به همراه آدرس داده مورد نظر صدا زده شود. سپس به حالت SendStart می‌رود و بیت آغاز فریم را روی پایه tx قرار می‌دهد. سپس در SendAddress آدرس را ارسال می‌کند و در SendStop بیت پایان فریم را می‌فرستد. سپس در حالت Wait منتظر صفر شدن پایه Rx توسط یواس‌بی می‌ماند و با صفر شدن آن، به حالت ReceiveData می‌رود و داده را دریافت می‌کند و با رفتن به حالت ReceiveStop بیت پایان فریم را دریافت می‌کند. در نهایت اگر بیت نهایی به درستی مقدار یک داشت، داده را روی بافر قرار می‌دهد و به حالت ابتدایی می‌رود و اگر هم قرار نداشت با چاپ پیام خطا به حالت ابتدایی می‌رود.

۸.۳ Sketchs

با تعریف Sketchها نحوه کارکرد میکروکنترلر و یواس‌بی را مشخص می‌کنیم. در اسکچ هارد دیسک در بدنه حلقه، مدام بررسی می‌کنیم که آدرس دریافت شده است یا نه. به محض دریافت شدن، داده را از متغیر m_storage می‌خوانیم و با فراخوانی write آن را به پروتکل می‌فرستیم.



شکل ۴: استیت دیاگرام میکروکنترلر

در میکروکنترلر هم در تابع setup در ابتدا یک آدرس را با فراخوانی write به پروتکل می‌فرستیم و در حلقه بررسی می‌کنیم هر وقت داده دریافت شد آن را چاپ می‌کنیم.

۹.۳ ارزیابی

برای دیباگ از رنگ‌های مختلفی برای چاپ استفاده کردیم به طوریکه تمام متون چاپ شده به رنگ قرمز متعلق به میکروکنترلر و متون به رنگ زرد متعلق به یواس‌بی هستند. همچنین هنگام فراخوانی تابع write توسط اسکچ هارد دیسک، مقدار متناظر با آن آدرس را چاپ می‌کنیم و به طور مشابه هنگام دریافت داده در اسکچ میکروکنترلر هم داده دریافت شده را چاپ می‌کنیم و با یکسان بودن این دو از پیاده‌سازی درست اطمینان حاصل می‌کنیم.

۴ پیاده‌سازی یک I2C Multiplexer

۱.۴ توضیحات مسئله

در این بخش هدف طراحی و پیاده‌سازی یک I2C Multiplexer است تا بتوان چندین سنسور را که از پروتکل I2C استفاده می‌کنند، به یک میکروکنترلر متصل کرد؛ حتی در حالتی که این سنسورها دارای آدرس یکسان باشند. همان‌طور که در بخش اول مشاهده شد، در پروتکل I2C تمام دستگاه‌ها روی یک باس مشترک قرار دارند و ارتباط بر اساس آدرس انجام می‌شود. در صورتی که چند سنسور دارای آدرس یکسان باشند، هنگام ارسال آدرس، همه آن‌ها به طور همزمان فعال شده و شروع به ارسال داده می‌کنند که این موضوع باعث تداخل روی خط SDA و دریافت داده نادرست توسط میکروکنترلر می‌شود.

برای حل این مشکل از یک واسطه به نام I2C Multiplexer استفاده می‌شود. این ماژول بین میکروکنترلر و سنسورها قرار می‌گیرد و وظیفه آن انتخاب یکی از سنسورها و اتصال آن به باس I2C است، به طوری که در هر لحظه فقط یک سنسور فعال باشد.

در این پروژه، ارتباط بین میکروکنترلر و مالتی‌پلکسر از طریق پروتکل USART برقرار می‌شود. میکروکنترلر شماره سنسور مورد نظر را از طریق USART ارسال می‌کند و مالتی‌پلکسر پس از دریافت این مقدار:

□ سنسور متناظر را فعال می‌کند.

□ از طریق پروتکل I2C داده آن سنسور را دریافت می‌کند.

□ داده دریافت‌شده را مجدداً از طریق USART برای میکروکنترلر ارسال می‌کند.

در نتیجه، این بخش ترکیبی از دو پروتکل USART و I2C است که به کمک آن می‌توان بدون ایجاد تداخل، چند سنسور با آدرس مشابه را مدیریت و داده‌های آن‌ها را به صورت انتخابی دریافت کرد.

۲.۴ سوالات:

۱.۲.۴ (۱) تابع reverse در Byte.h و کارکرد آن

تابع reverse در فایل Byte.h یک بایت را دریافت کرده و ترتیب بیت‌های آن را معکوس می‌کند. پیاده‌سازی آن به این صورت است که از بیت با ارزش مکانی کمتر (000) شروع کرده و بیت‌ها را یکی یکی جدا می‌کند و در یک متغیر جدید (res) که ابتدا صفر است، قرار می‌دهد. سپس res را یک واحد به چپ شیفت می‌دهد تا بیت بعدی در جای مناسب خود قرار گیرد.

در پروژه، از این تابع در کلاس‌های پروتکل I2C برای بررسی آدرس استفاده شده است (برای مثال در VL530X.h). آدرس ارسال شده توسط میکروکنترلر به صورت معکوس دریافت می‌شود و سنسور با معکوس کردن آن، بررسی می‌کند که آیا با آدرس خودش (0x29) مطابقت دارد یا خیر.

آیا در بردهای واقعی استفاده می‌شود؟ این عملیات معکوس کردن بیت‌ها به خودی خود یک عملیات استاندارد سخت‌افزاری نیست. در پروتکل‌های واقعی، آدرس به همان ترتیبی که ارسال می‌شود دریافت و تفسیر می‌گردد. مشکلی که در این شبیه‌ساز با آن مواجه هستیم به دلیل نحوه‌ی پیاده‌سازی انتقال بیت‌ها و ذخیره‌سازی آن‌ها است.

راه حل دنیای واقعی: در یک سیستم واقعی، با هر پالس کلاک (SCL)، وضعیت خط SDA خوانده می‌شود و مستقیماً به عنوان بیت مربوطه تفسیر می‌گردد. اگر یک بایت به صورت سریال و با شروع از باارزش‌ترین بیت MSB-First ارسال شود، گیرنده نیز آن را به همان صورت MSB-First بازسازی می‌کند. بنابراین، نیازی به معکوس کردن نیست، زیرا پروتکل استاندارد I2C دقیقاً مشخص می‌کند که MSB-First است. مشکل اصلی در این شبیه‌ساز، تطبیق نحوه‌ی سریال‌سازی نرم‌افزاری با انتظارات است. راه حل عمیق این است که یا شیفت رجیستر نرم‌افزاری را به گونه‌ای پیاده‌سازی کنیم که بیت‌ها را به ترتیب صحیح دریافت کند، یا از ابتدا پروتکل را بر اساس یک ترتیب مشخص (مثلاً MSB-First) طراحی کرده و توابع read و write را طوری بنویسیم که این ترتیب را رعایت کنند.

۲.۲.۴ (۲) تفاوت دو تابع takeNthBit در Bit.h

در فایل Bit.h دو نسخه از تابع takeNthBit وجود دارد. تفاوت اصلی آن‌ها در زمان ارزیابی است:

۱. نسخه زمان اجرا (Runtime):

```
template <typename T>
requires std::is_integral_v<T>
inline constexpr Bit
takeNthBit(T value, std::uint8_t n)
```

این تابع یک مقدار (value) و یک اندیس (n) را به عنوان آرگومان در زمان اجرا دریافت می‌کند. این تابع برای مواقعی استفاده می‌شود که مقدار n در زمان کامپایل مشخص نیست و ممکن است متغیر باشد.

۲. نسخه زمان کامپایل (Compile-Time):

```
template <auto byte, std::uint8_t n>
requires std::is_integral_v<decltype(byte)>
inline constexpr Bit
takeNthBit()
```

این تابع هیچ آرگومانی دریافت نمی‌کند و هر دو پارامتر مقدار (byte) و اندیس (n) به عنوان آرگومان‌های template به آن داده می‌شوند که باید در زمان کامپایل مشخص باشند. مزیت این نسخه این است که می‌تواند بررسی‌های ایستا (static_assert) انجام دهد و از آن در محاسبات constexpr استفاده شود.

۳.۲.۴ (۳) تابع attachPinToCommunicationClock در Board

توضیح از جهت کد: این تابع یک متد template در کلاس Board است. یک اندیس (pinIndex) دریافت می‌کند و از طریق آن به یکی از اعضای Gpio که از نوع Pins::Digital است دسترسی پیدا می‌کند. این کار با استفاده از boost::pfr::get<pinIndex>(m_gpio) انجام می‌شود. سپس، یک اتصال (Connection) بین سیگنال communicationClockChanged پردازنده (m_processor) و یک تابع لامبدا برقرار می‌کند. این لامبدا هر بار که کلاک پردازنده تغییر وضعیت دهد (لبه‌ی کلاک)، وضعیت آن لبه (edge Bit) را دریافت کرده و با فراخوانی pin.write(edge) روی پایه‌ی دیجیتال مشخص‌شده، همان لبه را بازتولید می‌کند. چندین static_assert در این تابع وجود دارد:

□ بررسی می‌کند که فرکانس برد Drived نباشد (یعنی خودش تولیدکننده کلاک باشد).

□ بررسی می‌کند که pinIndex معتبر باشد.

□ بررسی می‌کند که نوع پایه حتماً Pins::Digital باشد.

کاربرد: کاربرد اصلی این تابع خودکارسازی اتصال پایه‌های کلاک (SCL) در پروتکل‌های ارتباطی نظیر I2C است. همان‌طور که در مستندات پروژه ذکر شده، "پایه SCL به صورت خودکار به کلاک میکروکنترلر متصل می‌شود". این تابع دقیقاً این اتصال خودکار را بین کلاک داخلی پردازنده و یک پایه GPIO مشخص برقرار می‌کند. با این کار، سیگنال کلاک مورد نیاز برای سنسورها و ماژول‌های جانبی بدون دخالت برنامه‌نویس در هر سیکل پردازنده تولید و روی لینک SCL قرار می‌گیرد. این کار همگام‌سازی (Synchronization) دقیق بین دستگاه‌ها را تضمین می‌کند و پیچیدگی ناشی از تولید دستی سیگنال کلاک در نرم‌افزار (Bit-Banging) را حذف می‌کند.

۳.۴ پیاده‌سازی

برای پیاده‌سازی این بخش، ماژول جدیدی به نام I2CMux طراحی کردیم که ترکیبی از دو پروتکل پیاده‌سازی شده در بخش‌های قبل (I2C و USART) است. معماری کلی سیستم به این صورت است که میکروکنترلر (MicroController3) شماره کانال مورد نظر را از طریق USART به I2CMux ارسال می‌کند. I2CMux پس از دریافت شماره کانال، پروتکل I2C مربوط به آن سنسور خاص را فعال کرده، داده را از سنسور دریافت می‌کند و سپس آن را مجدداً از طریق USART به میکروکنترلر بازمی‌گرداند.

نکته مهم در طراحی سخت‌افزار مجازی: در این بخش، سه برد Esp8266، I2CMux و دو سنسور V1530x را از طریق لینک‌های مجازی به هم متصل کردیم. نکته کلیدی این است که I2CMux دارای دو پورت مجزای I2C است (sda1/sc11 و sda2/sc12) که هر کدام به یک سنسور متصل شده‌اند. این طراحی دقیقاً مشکل "آدرس یکسان" را حل می‌کند، زیرا در هر لحظه فقط یک باس I2C فعال است.

کلاس I2CMux و ماشین حالت آن

این کلاس که در پوشه Hardwares/Boards قرار دارد، یک برد پردازنده‌ای مستقل با دو پروتکل I2C و یک پروتکل USART است. ماشین حالت I2CMux در فایل I2CMultiplexer.h (بخش Sketch) پیاده‌سازی شده و شامل چهار حالت اصلی است:

۱. **R_USART (دریافت از USART):** در این حالت، مالتی‌پلکسر منتظر دریافت یک بایت از میکروکنترلر از طریق پروتکل USART است. هرگاه داده‌ای دریافت شد، این بایت به عنوان شماره کانال (m_channel) ذخیره شده و پروتکل I2C متناظر با آن کانال (i2c0 برای کانال ۰ و i2c1 برای کانال ۱) با آدرس 0x29 مقداردهی اولیه می‌شود. سپس به حالت I2C می‌رویم.

۲. **I2C (ارتباط با سنسور):** در این حالت، پروتکل I2C که در بخش اول تمرین نوشته بودیم، اجرا می‌شود. از کلاس ByteStream برای جمع‌آوری دو بایت داده از سنسور استفاده می‌کنیم تا یک عدد uint16_t کامل تشکیل شود. پس از تکمیل داده، به حالت W_USART1 می‌رویم.

۳. **W_USART1 و W_USART2 (ارسال به USART):** داده دریافتی که یک عدد ۱۶ بیتی است، باید در دو بایت جداگانه برای میکروکنترلر ارسال شود. بنابراین، با استفاده از تابع <N>getBytes (که در بخش اول توضیح داده شد)، بایت بزرگتر (<1>getBytes) و سپس بایت کوچکتر (<0>getBytes) را استخراج کرده و هر کدام را طی یک فریم مجزای USART به سمت میکروکنترلر ارسال می‌کنیم. پس از ارسال هر دو بایت، به حالت اولیه بازمی‌گردیم.

تغییرات در MicroController3

اسکچ میکروکنترلر (MicroController3.h) نیز به‌روزرسانی شد. این اسکچ در ابتدای کار (setup) شماره کانال ۰ را برای فعال‌سازی سنسور اول از طریق USART ارسال می‌کند. سپس در حلقه اصلی (loop)، منتظر دریافت پاسخ از مالتی‌پلکسر می‌ماند. از آنجایی که پاسخ یک عدد ۱۶ بیتی است، از کلاس ByteStream برای سرهم کردن دو بایت دریافتی از USART استفاده می‌شود تا دوباره به عدد اصلی تبدیل گردد.

نحوه استفاده مجدد از کدهای قبلی

بخش زیادی از کد مورد نیاز برای این قسمت، با استفاده مجدد از پروتکل‌های پیاده‌سازی شده در بخش‌های ۱ و ۲ تأمین شد:

□ **پروتکل I2C:** کلاس‌های I2C که در بخش اول برای Esp8266 و V1530x نوشتیم، بدون تغییر در I2CMux نیز استفاده شدند. این نشان‌دهنده طراحی ماژولار پروتکل‌هاست.

□ **پروتکل USART:** مشابه بخش دوم، از پروتکل USART پیاده‌سازی شده هم در سمت Esp8266 و هم در سمت I2CMux برای تبادل فرمان و داده استفاده کردیم. منطق Full-Duplex بودن آن به ما این امکان را داد که I2CMux بتواند همزمان فرمان جدیدی دریافت نکند و روی ارسال داده تمرکز کند.

□ **کلاس‌های کمکی:** کلاس ByteStream برای اسمبل کردن بایت‌ها و توابع getByte برای جداسازی بایت‌ها، از ابزارهای کلیدی بودند که در تمامی بخش‌ها استفاده شدند.

۴.۴ ارزیابی

برای اطمینان از درستی پیاده‌سازی، عملکرد کلی سیستم را با هر دو سنسور (کانال ۰ و ۱) آزمایش کردیم. در MicroController3 در تابع setup ابتدا دستور خواندن از کانال ۰ صادر می‌شود. پس از دریافت و چاپ داده، اسکچ را به صورت دستی تغییر داده و کانال ۱ را درخواست کردیم. در هر دو حالت، داده چاپ شده در میکروکنترلر را با داده تولید شده توسط سنسور متناظر (که با رنگ سبز در کنسول مشخص بود) مقایسه کردیم و از یکسان بودن آن‌ها اطمینان حاصل کردیم.

همچنین صحت مسیر انتقال داده را از طریق لاگ‌های رنگی رهگیری کردیم. میکروکنترلر با رنگ قرمز، مالتی‌پلکسر با رنگ آبی و سنسورها با رنگ سبز لاگ خود را چاپ می‌کردند. بررسی این لاگ‌ها نشان داد که ترتیب عملیات دقیقاً مطابق انتظار است: میکروکنترلر شماره کانال را ارسال می‌کند، مالتی‌پلکسر آن را دریافت کرده، سنسور مربوطه فعال شده و داده را برمی‌گرداند و نهایتاً مالتی‌پلکسر آن را به میکروکنترلر فوروارد می‌کند. این روند برای هر دو کانال با موفقیت انجام شد.